

Task Definition and Acceptance Criteria

The agentic tool I am using for this assignment is Claude Code CLI set with Sonnet 5 at High effort. The feature that I will delegate is input validation & error handling. My prompt that I will give the agent will be

Before writing any code, look through the codebase and make sure you understand the current progress of the files. Do not edit any files until I approve, we are currently in plan mode. My instructions are as following add input validation and error handling to task_tracker.py, working on the branch feature/agentBranch. Make sure that

1. It can validate titles on 'add'. Reject the title if it is empty, just whitespace, or has any non ASCII character (in other words, only allow printable ASCII characters such as letters, numbers, spaces, and punctuation). When rejected, print a clear error, return non zero from main(), and don't write to the file.

2. It can handle a corrupt data file. If the JSON is corrupted, show a short error and exit non zero instead of crashing. A missing file should still give an empty list.

3. My two acceptance tests are at the bottom of test_task_tracker.py, do not modify them. They are test_add_rejects_empty_title and test_add_rejects_non_ascii_title.

Once we have agreed on a plan, then follow through with my instructions. Do all of your work on the current branch feature/agentBranch.

Checkpoint Records

My checkpoints will be approving a plan before any code and a full diff review before the merge. The plan that Claude Code gave me is in the appendix at the end of this document.

So, at my first checkpoint, I had the agent propose a full implementation plan in plan mode and made no edits until I reviewed it. Overall the plan looks great and I approved it. It scoped itself to the two requirements I gave which were title validation on add and corrupt file handling. It did not create additional features such as a maximum title length so there was no scope creep to fix. It kept all of the validation and error handling in the main() and left the core functions unchanged. This kept the already existing test cases from breaking. Claude also identified my two additional tests that I created and didn't touch them either. One thing I notice was that the plan added isprintable() along with the isascii() so something like tabs or newlines would also be rejected. This goes slightly further than my prompt, so therefore it is a small example of the agent making a judgement call I did not exactly say. However, I think it's a very acceptable addition and left it in. The plan also adds a new test of its own for the corrupt file criteria. I also kept it due to it being a reasonable addition, but this is an agent test, not one I created. My two tests that I created stayed untouched.

Code Review Comments

I reviewed the full diff and ran `python -m pytest test_task_tracker.py` to ensure all tests passed, and yes, I had 12 tests pass. The original 9 passed, my 2 additional tests passed, as well as the last test that the agent added passed.

1. `Validate_title()` is clean and well created. The logic is pulled into a function that returns an error message or none, rather than being inlined into main. This is the right structure and I'd approve it.
2. Non list JSON is not handled, only a malformed JSON. The try/except catches `json.JSONDecodeError`, but a data file containing a valid JSON that isn't a list passes through the guard. This then causes it to fail later in code that would eventually expect a list.
3. `isprintable()` broadens the code beyond what I requested so it is beyond correctness/scope creep. It added a rejection to characters such as tabs and newlines. I'd argue it is a good thing to add, even though it was an unprompted expansion of the code. Secondly, I'd argue that it aligns with the purpose of the added feature.
4. The corrupt file test asserts too poorly and I'd say its a test quality issue.
`Test_main_handles_corrupt_data_file` only checks `rc != 0`, so therefore it doesn't confirm the error reached `stderr`, just that the return code was non zero. This could lead to a future change failing in a different way.

After approving the diff, which was one of my defined checkpoints, I organized the agent's work into two commits on `feature/agentBranch`. I also attributed the AI with an "assisted by" description. I committed my two acceptance tests separately before the agent's work (7dd235a), so the git history shows the spec was defined first, and then the agent implemented it. I then merged the branch into main and confirmed all 12 tests still passed post merge.

Reflection

My two checkpoints were plan approval before any code and a full diff review before the merge to the main branch. I would say both were very helpful. The plan approval checkpoint is where I caught the agent's decision to add an `isprintable()` check. The diff review checkpoint is where I found that corrupt file handling only caught `json.JSONDecodeError`, and would still break on a valid JSON file that wasn't a list. In terms of module failure modes, I observed mild scope creep (`isprintable`) and a confident wrong turn (`JSONDecodeError`). The `isprintable()` broadens the code beyond what I requested so it is beyond correctness/scope creep. It added a rejection to characters such as tabs and newlines. I'd argue it is a good thing to add, even though it was an unprompted expansion of the code. Secondly, I'd argue that it aligns with the purpose of the added feature. For the confident wrong turn, the `try/except` catches `json.JSONDecodeError`, but a data file containing a valid JSON that isn't a list passes through the guard. This then causes it to fail later in code that would eventually expect a list. One thing I didn't see was test gaming. The agent did not game my tests, leaving my two tests that I wrote untouched. Also, it added its own test (`test_main_handles_corrupt_data_file`) rather than weakening mine to pass. I also kept it due to it being a reasonable addition, but this is an agent test. Reviewing AI generated code felt different from reviewing my own or a classmates due to there not being a lot of context given. With the agentic coding, I had to treat every line as unexplained and had to verify them against the requirements. It made me more skeptical than I would be over one of my teammates' PR. I would trust this workflow on a real team only under the agent working on a separate branch. Secondly, I would require a human to read it throughout before merging. Along with these choices, a real full diff of the code would need to be done before merging. Given these things, the AI did the task in minutes which would have taken me

much longer to complete. Just like this assignment, I would still want a human to write acceptance tests that the agent cannot touch. This would ensure that the agent sees a goal and successfully meets it.

Appendix: Prompt and Transcript

Before writing any code, look through the codebase and make sure you understand the current progress of the files. Do not edit any files until I approve, we are currently in plan mode. My instructions are as following add input validation and error handling to `task_tracker.py`, working on the branch `feature/agentBranch`. Make sure that

1. It can validate titles on 'add'. Reject the title if it is empty, just whitespace, or has any non ASCII character (in other words, only allow printable ASCII characters such as letters, numbers, spaces, and punctuation). When rejected, print a clear error, return non zero from `main()`, and don't write to the file.

2. It can handle a corrupt data file. If the JSON is corrupted, show a short error and exit non zero instead of crashing. A missing file should still give an empty list.

3. My two acceptance tests are at the bottom of `test_task_tracker.py`, do not modify them. They are `test_add_rejects_empty_title` and `test_add_rejects_non_ascii_title`.

Once we have agreed on a plan, then follow through with my instructions. Do all of your work on the current branch `feature/agentBranch`.

Read 2 files, ran 1 shell command

- Updated plan

/plan to preview

- User approved Claude's plan

Plan saved to: ~/.claude/plans/before-writing-any-code-playful-squirrel.md · /plan to edit

Add input validation & error handling to task_tracker.py

Context

task_tracker.py currently trusts its inputs completely: add_task() accepts any string as a title (including empty/whitespace-only or non-ASCII text), and load_tasks() will crash with an uncaught json.JSONDecodeError if tasks.json is corrupted. The goal is to make the CLI fail gracefully and predictably in both cases, without changing any other existing behavior (all current tests in test_task_tracker.py must keep passing unmodified, including the two protected acceptance tests at the bottom of the file: test_add_rejects_empty_title and test_add_rejects_non_ascii_title).

Approach

1. Title validation on add

Add a small pure helper in task_tracker.py:

```
def validate_title(title):
    """Return an error message if title is invalid, else None."""
    if not title.strip():
        return "Title cannot be empty or whitespace only."
    if not title.isascii() or not title.isprintable():
        return "Title must contain only printable ASCII characters."
    return None
```

- title.strip() empty → catches empty string and whitespace-only titles (satisfies

test_add_rejects_empty_title).

- isascii() → catches any non-ASCII character, e.g. "fiancé" (satisfies

test_add_rejects_non_ascii_title).

- isprintable() → also rejects embedded control characters (tabs/newlines), consistent with "only printable ASCII characters."

In main(), in the "add" branch, call validate_title(args.title) before add_task/save_tasks. If it returns an error message, print Error: {message} to stderr and return 1 immediately —

save_tasks is never reached, so no file is written.

2. Corrupt/missing data file handling

Leave load_tasks() itself unchanged (still returns [] when the file is missing — already correct).

In main(), wrap the existing tasks = load_tasks() call:

try:

```
tasks = load_tasks()
```

except json.JSONDecodeError:

```
print(f"Error: {DATA_FILE} contains invalid JSON.", file=sys.stderr)
```

```
return 1
```

This keeps `load_tasks()` a simple, testable function while making `main()` (the actual CLI entry point) exit non-zero with a short message instead of crashing with a traceback.

3. Tests

- Do not touch `test_add_rejects_empty_title` or `test_add_rejects_non_ascii_title`.
- Add one new test (new function, appended after the protected pair) that writes garbage to a temp data file and asserts `tt.main(["list"])` (or `["add", "x"]`) returns non-zero instead of raising — covers requirement #2 since no acceptance test currently exercises it.
- Run `python -m pytest test_task_tracker.py -v` to confirm the full suite (baseline + acceptance + new corrupt-file test) passes.

Files to modify

Verification

1. `python -m pytest test_task_tracker.py -v` — all tests pass.
2. Manual sanity checks on branch `feature/agentBranch`:
 - `python task_tracker.py add " "` → prints error, exit code 1, `tasks.json` not created/modified.
 - `python task_tracker.py add "fiancé"` → prints error, exit code 1.
 - `python task_tracker.py add "Buy milk"` → succeeds normally.
 - Write invalid JSON into a temp `tasks.json`, run `python task_tracker.py list` → short error message, exit code 1, no traceback.
 - Delete `tasks.json` entirely, run `python task_tracker.py list` → empty output, exit code 0.
- Auto mode lets Claude handle permission prompts automatically — Claude checks each tool call for risky actions and prompt injection before executing. Actions Claude identifies as safe are executed, while actions Claude identifies as risky are blocked and Claude may try a different approach. Ideal for long-running tasks. Sessions are slightly more expensive. Claude can make mistakes that allow harmful commands to run, it's recommended to only use in isolated environments. Shift+Tab to change mode.

Now implementing the approved plan.

- Update(`task_tracker.py`)

Added 9 lines

```
return max((t["id"] for t in tasks), default=0) + 1
```

```
def validate_title(title):
    """Return an error message if title is invalid, else None."""
    if not title.strip():
        return "Title cannot be empty or whitespace only."
    if not title.isascii() or not title.isprintable():
        return "Title must contain only printable ASCII characters."
    return None
```

```
def add_task(tasks, title, tag=None):
    """Add a new task and return it."""
    task = {
```

- Update(task_tracker.py)

Added 10 lines, removed 1 line

```
sub.add_parser("stats", help="Show summary statistics")
```

```
args = parser.parse_args(argv)
```

```
tasks = load_tasks()
```

```
try:
```

```
    tasks = load_tasks()
```

```
except json.JSONDecodeError:
```

```
print(f'Error: {DATA_FILE} contains invalid JSON.', file=sys.stderr)

return 1
```

```
if args.command == "add":

    error = validate_title(args.title)

    if error:

        print(f'Error: {error}', file=sys.stderr)

        return 1

    task = add_task(tasks, args.title, args.tag)

    save_tasks(tasks)

    print(f'Added task {task['id']}: {task['title']}")
```

Now adding the corrupt-file test after the two protected acceptance tests.

- Update(test_task_tracker.py)

Added 9 lines, removed 1 line

```
monkeypatch.setattr(tt, "DATA_FILE", str(data_file))

rc = tt.main(["add", "fiancé"])

assert rc != 0

assert not data_file.exists() or tt.load_tasks() == []
```

No newline at end of file

```
assert not data_file.exists() or tt.load_tasks
```

```
def test_main_handles_corrupt_data_file(tmp_path, monkeypatch):
```

```
data_file = tmp_path / "tasks.json"

data_file.write_text("{not valid json}")

monkeypatch.setattr(tt, "DATA_FILE", str(data_file))

rc = tt.main(["list"])

assert rc != 0
```

No newline at end of file

Now let's run the test suite and a few manual sanity checks.

Ran 3 shell commands

All 12 tests pass. Now some manual sanity checks with a scratch data file.

Ran 1 shell command

All behaviors match the plan exactly: rejected titles don't write the file, valid titles work normally, corrupt JSON exits cleanly with an error, and a missing file yields an empty list.

Cleaning up the scratch file.

Ran 1 shell command

Implementation is done on feature/agentBranch and verified:

- task_tracker.py: added validate_title() (rejects empty/whitespace-only and non-ASCII/non-printable titles), wired into the add branch of main() to print an error and return 1 without writing the file; wrapped load_tasks() in main() to catch json.JSONDecodeError and exit 1 with a short message instead of crashing.
- test_task_tracker.py: your two acceptance tests are untouched; added one new test, test_main_handles_corrupt_data_file, to cover the corrupt-file requirement.
- Full suite: 12/12 passing. Manual checks confirm no file is written on rejected titles, corrupt JSON exits cleanly, and a missing file still yields an empty list.

Not committed — let me know if you'd like these changes committed/pushed.

* Cogitated for 2m 7s